

Architectural Support for the Bend Programming Language

Edouard Clocheret¹, Ian Gordon¹, Julian Beaudry¹

¹Brown University

May 5, 2025

1 Abstract

Software and hardware are on a long journey of coevolution. Engineers design architectures to both support the programs of today and attempt to stay one step ahead of ever-vacillating software trends of tomorrow. Meanwhile developers create applications that try to best-utilize existing hardware while also pushing its capabilities. It is difficult for either party to predict where the industry will end up next.

This is best illustrated by modern graphics cards. Graphics cards were originally produced for scientific computing, hobbyist gaming, and personal media consumption. In the last few years, however, the unique ability to perform both highly parallel and advanced mathematical tasks has made GPUs a vital resource in the deep learning domain. Deep learning has become such a dominant application that companies prioritize its support in their GPU architectures, with special cores and math operations designed specifically for that use case.

On the other hand, with the end of Moore’s law, hardware designers have begun to expand outwards — increasing the core-count of computers rather than advancing the speed or transistor density. To create faster programs or support larger applications, software engineers have had to adapt and update their software to take advantage of parallelization. Just last year, the Python team decided to add an option to remove the GIL, allowing true multithreading.

Bend is a relatively new programming language that aims to capitalize on both modern trends. It is a high-level, massively parallel programming language designed to feel like but scale like CUDA. Its selling point is that you don’t have to do anything to make it parallel; if it can run in parallel, it will run in parallel. It works similarly to the OOO and ROB-based CPU architectures that we saw in class, by tracking dependent operations and attempting to run everything in parallel otherwise. It does this by maintaining an “Interaction Net” representing the entire program and running the program by performing parallel “reductions” all over the net. To learn more, ‘Zicklag’ has a great blog post on the topic. Or, of course, you could read the original paper.

Bend can operate on both the CPU and GPU, but the underlying architecture on which Bend runs is extremely important. Due to the way that it takes advantage of parallelism and massive number of threads, programs that fully utilize the language are the most performant when run on a GPU or a CPU with a large core count. At a high level, our goal is to evaluate Bend on a range of these platforms (CPUs and GPUs) and compare its performance to other high level languages. Given the results of that analysis, we aim to identify architectural features that are the most important for running Bend programs. In other words, answer the question “If you were to design a

computer architecture for running Bend-like programs, what would it look like?”.

As a fun exercise, see where Bend falls on the McMillen checklist.

1.1 Hypotheses

Hypothesis #1

Bend performs best on highly parallel recursive algorithms.

We hypothesize that Bend will show the greatest performance benefits on recursive algorithms with high task-level parallelism (e.g., Merge Sort, Depth-First Search), due to its implicit threading and graph-based execution model. These workloads should allow Bend to fully utilize its automatic parallelization features.

Hypothesis #2

Newer GPU architectures yield better Bend performance.

We expect that Bend’s performance will improve on newer GPU architectures (Ampere, Volta) compared to older ones (Pascal, Turing), due to increased core counts, memory bandwidth, and more efficient schedulers. We hypothesize that these hardware differences will directly correlate with reductions in execution time for compute-heavy benchmarks.

Hypothesis #3

Bend is less optimized for SIMD-style vector workloads.

We hypothesize that Bend will perform worse than hand-optimized CUDA or established GPU libraries (e.g., cuBLAS) on vectorized operations like matrix multiplication or image convolution. These tasks benefit from low-level tuning and explicit control over memory access patterns, which Bend abstracts away in favor of general-purpose parallelism.

Hypothesis #4

Bend programs use more memory than their Python counterpart.

We hypothesize that Bend programs, particularly recursive ones, will generally use more memory than their Python equivalents. This is due to Bend’s design, which focuses on maximizing parallelism wherever possible. The increased parallelism is expected to result in higher memory overhead, as additional resources are allocated to support the parallel execution.

2 Application

2.1 Traits and Use Case of Bend

This project explores the performance of the Bend programming language, a functional language designed around immutability and parallelization, across various GPU/CPU architectures to iden-

tify the optimal setup for executing Bend programs. Bend’s syntax is quite similar to Python (e.g., functions are defined with `def`, and explicit type declarations are unnecessary), but it enforces immutability, meaning recursion and other functional programming techniques are essential for developing efficient solutions.

2.2 Installing Bend

Getting started with Bend was surprisingly easy; you simply follow the installation instructions provided in the GitHub README.md. To summarize:

1. Clone the Bend repository from GitHub.
2. Follow the installation steps in the README.md file.
3. Install the necessary libraries and tools (rust and hvm)
4. Run example programs to verify the installation works correctly.
5. Use the provided shell scripts on this repo to run performance benchmarks on different GPU setups. In order to see the Bend, Python, and Cuda programs we used to benchmark, you can look in the `bend-scripts`, `python-scripts`, or `cuda-scripts` folder in the repository. Additionally, we made some other Bend scripts in this repo we used to test Bend’s performance.

2.3 Programming in Bend

While the Bend installation itself went smoothly, writing and analyzing Bend programs was a challenge. As a relatively new programming language, Bend has limited coding examples available, making it difficult to find references for implementing certain functionality. For instance, we couldn’t find an existing Bend implementation of matrix multiplication. This lack of pre-built functions meant that, in order to run one of our benchmark programs—a single-layer neural network—we had to implement basic matrix operations from scratch, along with other fundamental operations, to get the network up and running. You can find the code for our neural network implementation [here](#).

In terms of performance evaluation, we ran benchmark programs on different Oscar GPUs and conducted some profiling using our MacBook’s M4 CPU. To execute a Bend program on an Oscar GPU, we created shell scripts designed to execute the program with varying input sets. An example of such a shell script can be found [here](#).

2.4 Running Bend and External Tooling

Running Bend, once you have a valid program, is not without its quirks either. Currently, Bend lacks the ability to print at intermediate points in the code (only the final return value of the main function is printed). This, along with the lack of a stack trace, can make debugging difficult. All data in Bend is fixed to a 24-byte precision, and, when values overflow, they do so without warning and are simply returned as they are. Finally, Bend currently has limited support for passing arguments to a Bend script via the terminal (it is only support on one of the non-GPU interpreters). The best solution we found for this

The issues detailed in the previous paragraph are enumerated in the documentation with promises to be resolved at a future date. However, besides the documented issues, there were also several runtime issues that we encountered and were unable to resolve:

1. When running Bend on any `quadrortx` node on OSCAR, programs return an `INVALID` value, and, based on the runtime of these programs, it seems they crashed early in their runtime. The same bend program will work fine on other GPUs — this may be an issue with the CUDA version of OSCAR rather than bend.
2. While the `neural_net.bend` written by Julian runs with the other Bend interpreters, it fails to run when using the CUDA interpreter, preventing us from analyzing its performance on a GPU.

Most of these challenges that we ran into were not terribly consequential for our experiments, but the lack of support for Nvidia’s `ncu` analysis tool was a major roadblock between us and our original goal. We originally envisioned having much more fine-grained data fine-grained analysis of how Bend manages threads, blocks, and memory on Nvidia GPUs. Generally, this is not difficult with Nvidia’s Nsight analysis tools: Nsight Systems and Nsight Compute. However, while we were able to get `nsys` functional, we were never able to get `ncu` to work (unfortunately, the more important of the two for detailed GPU resource monitoring). As a result, our analysis and experiments are based off of timing and measuring RAM using (when running on CPUs).

We would like to thank the developers for their help in the process.

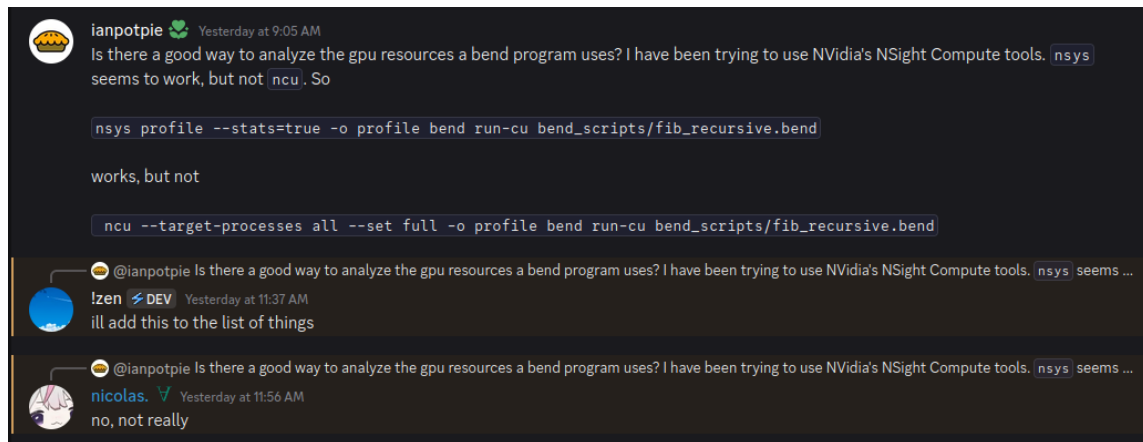


Figure 1: Developers Supporting Developers

3 Experimentation Environment

The first approach in order to measure a potential performance boost is execution time. For this part of the project, five different tasks were selected in order to compare the relative performance of three CPUs over the variety of tasks.

3.1 Architectures

The 3 architecture that are compared are:

- NVIDIA Ampere architecture, with a system that uses a 64-bit x86 architecture implemented by an AMD EPYC 7532 32-Core Processor across two sockets (totaling 64 physical cores without SMT), supporting 32-bit and 64-bit operating modes, little-endian byte order, AMD-V virtualization, and an extensive instruction set including AVX2, FMA, BMI1/2.

It appears to be well-suited for high-performance, parallel, and virtualized workloads in a server or HPC environment.

- The NVIDIA Turing architecture, with a system that using a 64-bit x86 architecture implemented by an Intel Xeon Gold 6242 CPU, featuring two sockets each with 16 physical cores (no hyperthreading), supporting 32-bit and 64-bit modes, 46-bit physical and 48-bit virtual addressing, and an advanced feature set including AVX-512 (with VNNI) and Intel VT-x virtualization.

It appears to be well-suited for high-throughput server or scientific workloads requiring strong floating-point and vector performance.

- The Apple M4 Pro personal computer. This system uses a 64-bit ARM-based architecture which features 14 CPU cores (10 high-performance and 4 energy-efficient cores) integrated with a unified 24 GB memory architecture.

It appears to be delivering high performance per watt optimized for macOS workloads including development, creative applications, and machine learning tasks.

3.2 Benchmark programs

The programs explored are:

- **Bitonic Sort:**

- Highly parallelizable due to its regular and predictable compare-exchange pattern, making it suitable for SIMD or GPU execution.
- Memory access is mostly predictable and local, which is cache-friendly.
- Expected to perform best on architectures with many cores or vector units (e.g., GPUs, parallel CPUs).
- High memory bandwidth usage during parallel stages due to concurrent reads and writes.

- **Graphic Application (Shader on an Image):**

- Very parallel, as each pixel can be processed independently, it is what GPUs are made for.
- Minimal memory usage per thread and relies on fast GPU memory and registers.
- Execution relies heavily on specialized hardware units such as vector ALUs.

- **Iterative Fibonacci:**

- Very low memory usage with only a few registers or stack variables needed.
 - Suitable for scalar CPU execution and can benefit from loop unrolling optimizations.
 - Predictable control flow, which improves branch prediction and instruction pipelining.
 - Not parallelizable, as each iteration depends on the previous.
- **Recursive Fibonacci:**
 - High memory usage due to deep call stacks and redundant computations.
 - Poor data locality and high register pressure on CPUs.
 - Control flow is irregular, making it harder for CPUs to predict branches and optimize instruction pipelines.
 - Parallelism is possible by computing recursive branches in parallel.
- **Single-Layer-Neural-Network:**
 - We wanted to test a more large-scale program to see how Bend performs in comparison to Python. By using a neural network, we introduced a more computationally intensive task that involves both forward and backward propagation, which includes matrix multiplications and gradient calculations.
 - Highly parallelizable due to the matrix multiplications, which are well-suited for SIMD (Single Instruction, Multiple Data) or GPU execution.
 - Memory usage is typically high in neural network computations due to the need for storing activations, gradients, and weights during the forward and backward passes. However, because we used the Iris flower dataset—a small dataset—we were able to keep memory usage at a minimum. This was crucial because Bend has strict memory limitations, and excessive memory usage can lead to program crashes.

3.3 Programming languages

In this study, the programs in Bend will be compared against their python counterparts. Also, Bend can be run in different settings.

- Python (interpreted): The program runs inside the Python virtual machine using a high-level interpreter on the CPU.
- Bend (single processor): The program executes through the Bend interpreter using a single CPU core without any parallelism.
- Bend using the C interpreter with parallelism: The Bend program is interpreted through a C backend that spawns parallel tasks on the CPU.
- Bend compiled to C with CPU parallelism: The Bend program is compiled to C, then compiled to a native binary that runs on multiple CPU cores using thread-based parallelism.
- Bend compiled to C with CUDA: The Bend program is compiled to C with CUDA support, enabling execution on GPU hardware with many parallel threads.

4 Evaluation

4.1 Experiment 1: A Bend Overview

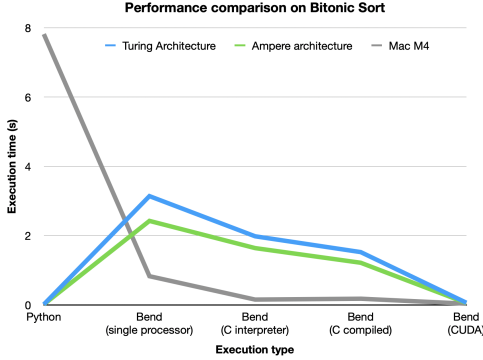


Figure 2: Bitonic sort

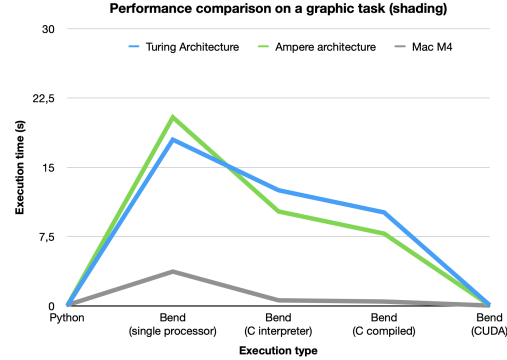


Figure 3: Shading task (graphic)

On these first two program benchmarks, Bitonic Sort and the image shading, the intended behavior of Bend is observed: as we go from the simplest version of Bend (interpreted, single processor), toward a compiled and parallel one, the execution time decreases. This behavior is observed on the 3 different architectures.

This trend reflects architectural expectations. Moving from interpretation to native compilation eliminates interpretation overhead, allowing tighter instruction scheduling, register allocation, and cache optimization. The addition of parallelism, especially for data-parallel workloads like Bitonic Sort and image shading, enables effective utilization of multi-core CPUs and SIMD or SIMT (in GPU) execution units. The consistency across architectures suggests that Bend’s compiler and parallel backend effectively map high-level parallelism to hardware resources (e.g., thread pools or GPU threads).

It also worth noticing that in these cases the switch to bend from python has a cost in performance for all architectures on the graphic task, and both GPUs on bitonic sort. The cost of running Bend is not neglectable until we turn on the access to the CUDA resources. The Mac has a slightly different behavior because it performs surprisingly slowly using python for the Bitonic sort, but benefits greatly from the switch to bend.

In its interpreted or CPU-only compiled forms, Bend likely introduces runtime overheads (e.g., dispatch logic, runtime memory management) not present in mature Python+NumPy or native CUDA implementations. Python often benefits from highly optimized C extensions or GPU bindings, which can outperform early-stage Bend execution. On GPUs, any abstraction overhead—such as delayed memory transfers or kernel launches—can significantly impact performance until true CUDA integration is enabled. The Mac’s slower Python performance could result from less optimized BLAS or NumPy backends, or from thermal throttling, making the relative performance gain from Bend more noticeable once compiled execution is used.

On the shading task (Figure 2), an interesting point is the comparison between the Turing and Ampere architecture. For the single processor, Ampere architecture is the best while for the multi processor the Turing architecture becomes better.

Ampere GPUs feature higher single-threaded performance and improved cache hierarchies, which can give an edge in single-core or low-parallelism workloads. However, Turing GPUs may have better warp-level scheduling or more mature driver/compiler support for certain parallel execution patterns, leading to superior multi-thread performance in highly parallelized tasks like shading. Additionally, Turing’s stream multiprocessor may be more effectively utilized by the specific code generated by Bend under CUDA, depending on how well the kernels map to hardware occupancy, shared memory usage, or warp divergence characteristics.

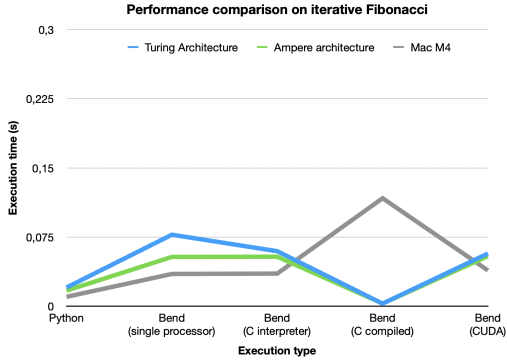


Figure 4: Iterative Fibonacci

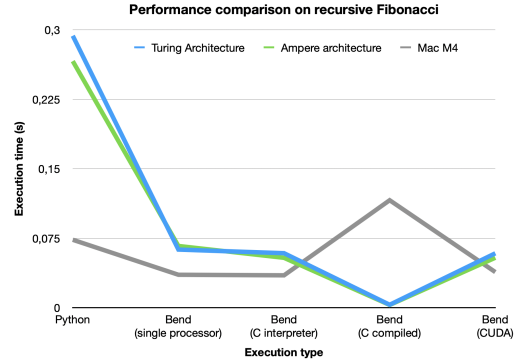


Figure 5: Recursive Fibonacci

These two benchmark focus on computing the Fibonacci series. The goal here is to explore how parallelism may improve the performance, between an iterative approach and a recursive approach. The recursive approach benefits from the possibility to be parallelized, computing independent branches of recursion independently, but at the cost of a lot of memory (for instance, past 30 iterations, the computation may crash).

Recursive Fibonacci exposes parallelism by allowing independent branches to compute simultaneously, which maps well to multi-threaded CPU or GPU execution—but it also causes deep call stacks and redundant memory allocations, leading to memory exhaustion on limited-memory architectures.

These results of execution time show that the recursive counterpart is comparable in Bend using parallelism, while slightly better on a single processor using a Turing architecture. When using python, the recursive version is on contrary extremely slower, due to memory considerations. In Python, recursion is hindered by interpreter overhead and limited stack size, making it drastically slower. In contrast, Bend with parallelism offloads branches efficiently across threads or cores, reducing latency, while Turing’s CPU handles the recursion tree slightly more efficiently than Ampere, possibly due to differences in CPU core clock or cache size.

On both versions of Fibonacci, the switch from python to Bend is enhancing performance when

we use the C compiler, except for the Mac that is never faster than when using python, and even has a bump in execution time when using the C compiler. Bend compiled to C offers lower-level control and better loop optimization, outperforming Python, except on Mac, where system-level differences (e.g., memory allocator, compiler backend, or thermal limits) may reduce the efficiency of compiled code versus Python’s highly optimized libraries.

For the iterative version (Figure 3), the Turing architecture is always worse than the Ampere architecture. Iterative Fibonacci benefits from strong single-thread performance and modern instruction pipelines. Ampere’s newer CPU/GPU microarchitecture and better branch prediction give it an edge over Turing for simple, loop-based tasks.

For the recursive version (Figure 4), exactly like for the image shading, the Turing architecture performs better than the Ampere architecture when it comes to single thread Bend, but not on multi-processor comparisons. Turing performs better in single-thread mode likely due to better handling of recursive function overhead or instruction caching, while Ampere scales better in multi-threaded runs thanks to higher core count, bandwidth, or improved task scheduling.

Conclusion

Based on the benchmark results and architectural interpretations, Ampere processors generally perform best for iterative, single-threaded, and memory-efficient tasks, thanks to their modern pipelines, better cache hierarchies, and improved branch prediction. Turing processors, on the other hand, show superior performance in multi-threaded and highly parallel workloads, particularly for recursive and shading tasks, likely due to more mature driver support and efficient warp-level scheduling. However, for compiled code leveraging CUDA, Ampere tends to scale better with parallelism, while Turing holds an advantage in single-threaded recursive and shading tasks. Platform differences, like those observed on the Mac, highlight that performance also depends on system-level factors such as memory handling and thermal constraints.

4.2 Experiment 2: GPU-Bound Program Analysis with nsys

Although we weren’t able to utilize Nvidia’s `ncu` tool as we originally planned, we still had access to the `nsys` tool, which provides information about system calls (both in the CPU and GPU) during the course of the program. Here is an example output for the `fib_recursive.bend` program being run with a value of 35 and utilizing CUDA on a Geforce 3090:

Executing 'osrt_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	Name
33.5	363,573,061	1	363,573,061.0	363,573,061.0	363,573,061	363,573,061	waitpid
30.3	328,895,596	25	13,155,823.8	1,180.0	300	318,473,165	read
16.6	180,626,954	1	180,626,954.0	180,626,954.0	180,626,954	180,626,954	sem_wait
10.8	117,060,243	14	8,361,445.9	1,633,816.5	960	39,076,444	poll
6.7	72,914,362	541	134,777.0	30,840.0	370	23,369,799	ioctl
1.3	14,444,559	61	236,796.0	5,380.0	880	12,041,196	open64
0.3	2,819,609	7	402,801.3	17,280.0	80	1,689,411	fread
0.2	1,825,278	53	34,439.2	430.0	170	1,138,047	fcntl
0.1	1,483,976	46	32,260.3	3,455.0	1,580	737,651	fopen
0.1	879,643	30	29,321.4	6,505.0	2,440	453,605	mmap64
0.1	618,794	5	123,758.8	100,369.0	59,890	192,958	pthread_create
0.1	568,074	13	43,698.0	15,950.0	820	397,456	write

0.0	134,558	20	6,727.9	3,320.0	1,090	40,770	mmap
0.0	74,599	66	1,130.3	50.5	40	51,950	fgets
0.0	42,021	39	1,077.5	920.0	540	4,069	fclose
0.0	39,161	6	6,526.8	6,115.5	1,280	15,480	open
0.0	36,140	12	3,011.7	3,185.0	2,120	4,220	munmap
0.0	22,040	2	11,020.0	11,020.0	11,000	11,040	pipe2
0.0	16,460	2	8,230.0	8,230.0	3,310	13,150	socket
0.0	12,020	1	12,020.0	12,020.0	12,020	12,020	connect
0.0	5,460	3	1,820.0	1,780.0	1,520	2,160	mprotect
0.0	4,720	1	4,720.0	4,720.0	4,720	4,720	fopen64
0.0	2,951	48	61.5	50.0	40	510	fflush
0.0	2,880	2	1,440.0	1,440.0	1,360	1,520	signal
0.0	2,820	1	2,820.0	2,820.0	2,820	2,820	bind
0.0	2,429	7	347.0	330.0	180	610	dup
0.0	2,260	8	282.5	230.0	160	520	sigaction
0.0	700	1	700.0	700.0	700	700	listen

Executing 'cuda_api_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	Name
92.3	127,171,900	2	63,585,950.0	63,585,950.0	1,395,784	125,776,116	cudaMemcpyToSymbol
5.2	7,218,549	5	1,443,709.8	49,999.0	15,790	4,490,990	cudaLaunchKernel
2.2	2,983,167	8	372,895.9	11,455.0	5,730	2,741,369	cudaMemcpy
0.3	425,516	1	425,516.0	425,516.0	425,516	425,516	cudaMalloc
0.0	5,890	1	5,890.0	5,890.0	5,890	5,890	cudaDeviceSynchronize
0.0	2,580	1	2,580.0	2,580.0	2,580	2,580	cuModuleGetLoadingMode

Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	Name
86.6	2,745,975	1	2,745,975.0	2,745,975.0	2,745,975	2,745,975	evaluator(GNet *)
10.6	334,495	1	334,495.0	334,495.0	334,495	334,495	initialize(GNet *)
2.7	86,079	1	86,079.0	86,079.0	86,079	86,079	print_result(GNet *)
0.1	1,792	1	1,792.0	1,792.0	1,792	1,792	boot_redex(GNet *, unsigned long long)
0.1	1,696	1	1,696.0	1,696.0	1,696	1,696	inbetween(GNet *)

Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	Operation
99.6	2,169,210	3	723,070.0	1,069,085.0	512	1,099,613	[CUDA memcpy HtoD]
0.4	9,504	7	1,357.7	1,376.0	1,152	1,760	[CUDA memcpy DtoH]

Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	Operation
51.380	3	17.127	25.690	0.000	25.690	[CUDA memcpy HtoD]
0.000	7	0.000	0.000	0.000	0.000	[CUDA memcpy DtoH]

There are a few takeaways here. For one, the runtime is dominated by the `waitpid` and `read` on the CPU. The time spent on these system calls overlaps with the time that the GPU spends calculating (the CPU must wait for the GPU to return with its result), but even with this in mind, the time spent on `evaluation(GNet *)`, where the the main calculations are performed is vastly outweighed by the time spent on CPU operations.

Our takeaway here is that small-to-mid sized program like our (intentionally unoptimized) recursive Fibonacci under-utilizes the GPU and the overhead of moving data to the CPU outweighs the speedup of performing those calculations on the GPU. For larger programs, like the graphic script or the recursive Fibonacci performed on larger inputs, we observed that the evaluation time on the GPU was much closer (within 10%) of the time spent on CPU-bound tasks.

Surprisingly, even for heavier programs (like our graphic script and recursive Fibonacci with

larger inputs) the amount of data transmitted to the GPU (captured by [CUDA `memcpy HtoD`]) does not far exceed 50MB. This tells us that the data sent to the GPU is likely proportional to the original program size, rather than being proportional to the amount of intermediate computation. This is surprising given the fact that, behind the scenes, Bend creates and manages an intermediate computation graph representing the program, which the GPU must then operate on. We expected data related to the computation graph to be sent to the GPU and scale with the amount of overall compute that a program required. It seems that this is not the case. This means that, in order to support a heavy “Bend workload”, a machine does not require high throughput between the GPU and CPU.

4.3 Experiment 3: Timing Bend GPU Programs

	a5000	a5500	geforce3090	quadrortx	titanrtx
Bitonic Sort	7.470	7.610	7.300	NA	NA
Fibonacci Iterative (43)	0.069	0.075	0.059	NA	NA
Fibonacci Recursive (43)	21.975	22.135	21.483	NA	NA
Graphic	132.664	134.025	131.314	NA	NA

Table 1: Program Runtimes by GPU (seconds)

The goal of the experiment was to provide more information about how Bend programs perform across different architectures and between GPUs on the same architecture. For this experiment, we prepared a script to run benchmarks across a grid of bend programs and a large set of GPUs listed as available under OSCAR. The original list of GPUs included those detailed in the table above, as well as the a6000, a40, and v100 GPUs. Unfortunately, while those GPUs were listed under OSCAR’s resources, they appeared unavailable upon request using `sbatch`. Likewise, while we were able to launch programs successfully on the quadrortx and titanrtx cards, they failed to produce an output and the overall runtime indicated that they had terminated early.

The remaining GPUs solely provide representation of the Ampere architecture, but show some consistent patterns across the cards, with the Geforce 3090 outperforming the other two, the a5000 coming in second, and last for the a5500. While all of the remaining GPUs utilize the same architecture, the GeForce 3090 features a higher boost clock speed (1.7 GHz compared to 1.65 GHz for the A5000/A5500) and significantly more CUDA cores (10,496 versus 8,192 for the A5000 and 10,240 for the A5500). These raw computational advantages likely outweigh the benefits of the professional cards’ ECC memory and enterprise-grade drivers for Bend workloads. Additionally, the 3090’s greater memory bandwidth (936 GB/s versus 768 GB/s for the A5000 and 768 GB/s for the A5500) could be providing an advantage when transferring data between the interaction net components that Bend relies on for parallel execution, explaining the consistent performance advantage across different program types. However, the fact that all three cards perform so closely and the fact that the a5500, which generally has more resources, outperforms the a5000, likely means that the Bend programs are not being significantly bottlenecked by the resources of the cards.

4.4 Experiment 4: Python vs Bend Runtime

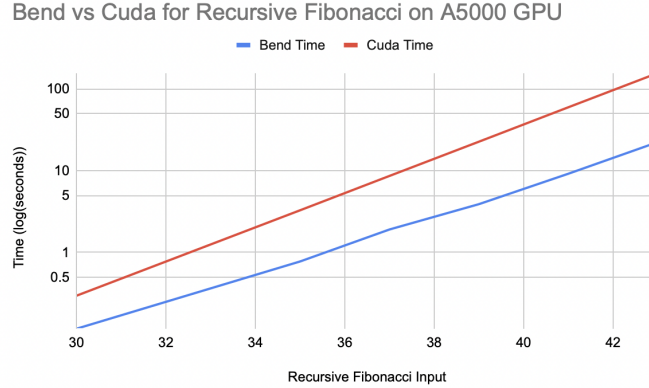


Figure 6: Runtime Comparison between Python and Bend Implementations

In order to compare Bend’s utilization of CUDA compared to a native implementation of CUDA, we benchmarked the Bend recursive Fibonacci script against our own implementation in CUDA. Without knowing too much about how CUDA functions behind the scenes, we expected the native implementation to vastly outperform the Bend implementation. We also expected the native CUDA code to compile and use some simplicity vectorization that the bend program would not be able to utilize. Although we expected the Bend implementation to underperform its CUDA counterpart, we had hopes that the behavior of both programs as we scaled the input would provide information about how both were utilizing resources.

Unexpectedly, as we can see from the results in the graph, the Bend implementation performs significantly better than the CUDA implementation. Since Bend utilizes CUDA under the hood, this certainly means that our CUDA implementation is lacking optimizations that we missed and does not perform implicit vectorization during compilation, as we speculated. This is important because it highlights a use case where Bend may be favorable for developers to use. If developers are not familiar with programming in CUDA (as we aren’t), they may opt to program in Bend because, for a naive implementation their Bend program could perform better.

4.5 Experiment 5: Python vs Bend Neural Network

As shown in the figure below, the neural network runtime in Python is significantly faster than in Bend. This outcome confirms the prediction made in Hypothesis 2, which suggested that Bend would struggle with SIMD-style workloads. The Python implementation benefits from the highly optimized numpy library for matrix operations, which is specifically designed for such tasks. In contrast, Bend abstracts away these low-level optimizations in favor of more general-purpose parallelism, which leads to poorer performance on tasks that are already well-optimized.

Another contributing factor is that the Bend program may not be as fine-tuned as it could be for matrix and vector operations. Due to our limited exposure to the Bend language and the lack of examples for matrix multiplication in the Bend documentation, we were forced to implement this functionality ourselves. This likely resulted in a suboptimal approach for handling such workloads.

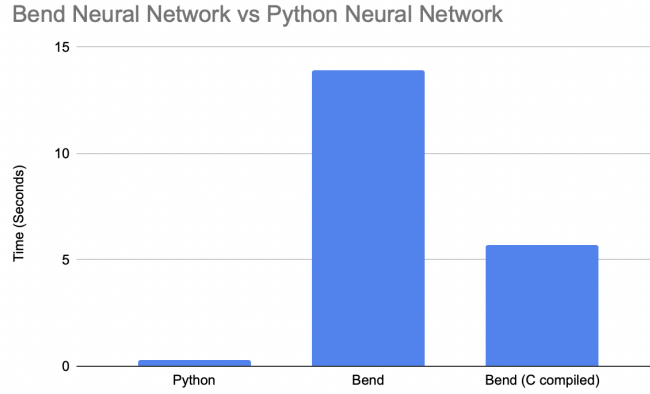


Figure 7: Neural Network Performance Bend vs Python

Ideally, we would have conducted this experiment using multiple GPUs and leveraged CUDA to assess which GPU would provide the best performance for the Bend implementation and compare it to Python. Unfortunately, we encountered difficulties running the Bend neural network program on CUDA, and were only able to execute it on Rust. As a result, we were unable to evaluate how the Bend program leverages Cuda.

4.6 Experiment 6: Python vs Bend Memory Consumption

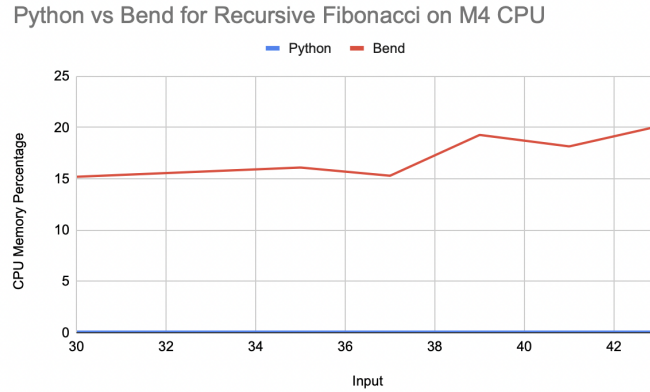


Figure 8: Memory Comparison between Python and Bend Implementations

As depicted in the figure above, we observe that the memory consumption of Bend is significantly higher than that of Python. This outcome aligns with the prediction made in Hypothesis 4, which suggested that the memory usage of Bend would be greater due to its inherent parallelization features.

The core reason for this increased memory usage in Bend lies in the language’s design, which aggressively attempts to parallelize operations. While parallelism can enhance performance, it also introduces additional overhead. In Bend, this overhead results in a higher memory footprint as the language manages multiple threads or processes simultaneously, each requiring its own stack and state. Consequently, memory usage grows considerably, even for simple recursive operations like Fibonacci sequence calculation.

In contrast, the Python implementation demonstrates minimal memory usage, consuming only around 0.1 percent of CPU memory. This is due to Python’s more conservative memory model, where the call stack typically does not grow beyond a depth of 41 for the recursive Fibonacci function. Since Python does not parallelize the recursion, the memory overhead is minimal—each recursive call simply adds a new frame to the call stack, which is efficiently managed and discarded when the recursion unwinds.

On the other hand, Bend’s approach, which involves parallelizing recursive calls, introduces significant memory overhead. The figure clearly shows that Bend’s memory usage hovers around 15-20 percent of CPU memory, a stark contrast to the minimal memory consumption observed in Python. This discrepancy highlights the tradeoff between parallelization and memory efficiency, where the potential performance benefits of parallelism come at the cost of increased memory usage.

Overall, this analysis supports our hypothesis that Bend’s aggressive parallelization strategy leads to higher memory consumption compared to Python’s more straightforward, non-parallelized recursive implementation.

5 Conclusion

Our benchmarks demonstrate that while Bend can outperform naive CUDA implementations in certain recursive workloads, thanks to its high-level abstractions and implicit parallelism, it struggles with highly optimized SIMD-style tasks like neural networks, where mature libraries like NumPy in Python excel. Additionally, Bend’s aggressive parallelization strategy results in significantly higher memory usage compared to Python, highlighting a tradeoff between performance and resource efficiency. These results suggest that Bend is a promising tool for developers seeking ease of parallelism without low-level CUDA programming, but it may not yet be optimal for memory-constrained or vectorized workloads. Given more time, we would have liked to deepen our understanding of the Bend programming language in order to write more optimized code. This would have allowed us to fully explore its capabilities and give it a fair opportunity to compete with other, more optimized languages.

6 References

- The Bend project repository
<https://github.com/HigherOrderCO/Bend/tree/main>
- Bend Programming Language Guide (Unofficial)
<https://bendlang.com/>
- Bend in X minutes - the ultimate guide
<https://github.com/HigherOrderCO/Bend/blob/main/GUIDE.md>

- Our Code Repositories

repo 1 (contains the Oscar shell scripts and Python, Cuda, and Bend scripts)

repo 2 (contains some Bend scripts (used for cross-compilation))